

Računarska vežba 8

PYTHON ALGORITMI PRETRAŽIVANJA

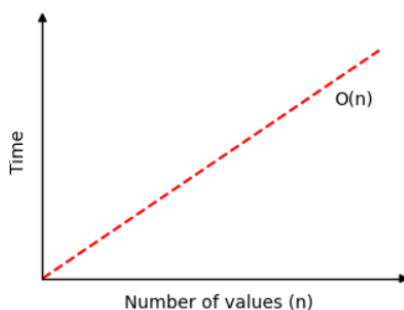
1 Algoritam linearnog pretraživanja

Linearno pretraživanje spada u klasu “brute force” algoritama, kod kojih se proverava svaki član sekvencijalno upisanih elemenata niza.

Linearno pretraživanje se primenjuje na nesortirani niz ili listu elemenata, pri čemu se utvrđuje da li se određeni element (ključ, key) nalazi u nizu.

- U prvom koraku polazi se od prvog elementa (tekućeg) koji se upoređuje sa ključem, ako prvi element ne sadrži vrednost ključa, prelazi se na drugi element niza
- U drugom koraku se drugi element niza poredi sa vrednošću ključa , a ako nisu jednaki, prelazi se na treći element niza
- Proces se ponavlja sve dok se u ne pronađe traženi broj na određenoj poziciji ili indeksu niza
- Šta ako traženi element nije u nizu? Vršiti se iteracija kroz celu sekvencu niza, element po element niza dok se ne pronađe traženi element ili dok se svi elementi ne ispituju i dođe se do poslednjeg u nizu

Vremenska složenost linearnog pretraživanja je $O(n)$:



Programski kod 1:

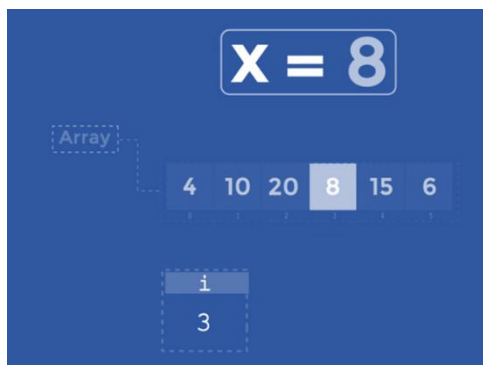
```
1 def lin_pretraga(niz, trazeni_broj):
2     for i in range(len(niz)):
3         if niz[i] == trazeni_broj:
4             return i
5     return -1
6
7 lista = [3, 17, 2, 9, 5, 1, 8, 4, 6, 34, 17, 7, 123, -3]
8 x = 7
9
10 rezultat = lin_pretraga(lista, x)
11
12 if rezultat != -1:
13     print("Element pronadjen na indeksu", rezultat)
14 else:
15     print("Element nije pronadjen")
16
```

- Element pronadjen na indeksu 11

Programski kod 2 (drugačija implementacija range()):

```
1 def linear_search(arr, n, x):
2     for i in range(0, n):
3         if (arr[i] == x):
4             return i
5     return -1
6 arr = [4,10,20,8,15,6]
7 x = 8
8 n=len(arr)
9
10 result = linear_search(arr,n,x)
11
12 if (result != -1):
13     print("Trazeni Element pronadjen na indeksu:", result)
14 else:
15     print("Element nije pronadjen!")
```

Trazeni Element pronadjen na indeksu: 3



Programski kod 3:

Korišćenjem prethodnih primera napisati programski kod za linearno pretraživanje elementa niza, koji će kao izlaz imati sledeće opcije.

```
Unesi veličinu liste: 20
Unesi donju granicu opsega: 300
Unesi gornju granicu opsega: 1000
Random lista brojeva: [439, 385, 628, 757, 442, 405, 367, 512, 428,
327, 963, 431, 695, 607, 597, 535, 775, 861, 915, 937]
Unesi broj koji treba da se pronade: 700
Broj 700 nije pronađen u listi.
```

Potrebno je uneti broj elemenata liste (niza) koji se pretražuje, donju i gornju granicu opsega niza i korišćenjem metode `random.randint()` omogućiti unos slučajnih brojeva u zadatom opsegu pretraživanja.

```
1  import random
2
3  def linear_recursive_search(arr, target, index=0):
4      if index >= len(arr):
5          return -1 # broj nije pronađen
6      if arr[index] == target:
7          return index # broj je pronađen
8      return linear_recursive_search(arr, target, index + 1)
9
10 # Unos za podatke veličine liste i opsega
11 user_size = int(input("Unesi veličinu liste: "))
12 user_min = int(input("Unesi donju granicu opsega: "))
13 user_max = int(input("Unesi gornju granicu opsega: "))
14
15 # Generisanje random liste na osnovu ulaznih podataka
16 random_list = [random.randint(user_min, user_max) for _ in range(user_size)]
17 print("Random lista brojeva:", random_list)
18
19 # Unos vrednosti broja koji se traži
20 user_target = int(input("Unesi broj koji treba da se pronade: "))
21
22 # Izvršavanje linearne rekurzivne pretrage
23 index_found = linear_recursive_search(random_list, user_target)
24 if index_found == -1:
25     print(f"Broj {user_target} nije pronađen u listi.")
26 else:
27     print(f"Broj {user_target} je pronađen na indeksu {index_found}.")
```

2 Algoritmi binarnog pretraživanja

Binarno pretraživanje je efikasan algoritam koji se koristi za pronalaženje određene vrednosti unutar sortiranog niza. Funkcioniše tako što više puta deli interval pretrage na pola dok se ne pronađe tražena vrednost ili interval ne postane prazan.

Pravilo stable binarnog pretraživanja: Sve vrednosti koje se nalaze u podstablu sa leve strane posmatranog čvora, moraju biti manje od vrednosti koja je pripisana datom čvoru, a sve vrednosti koje se nalaze u desnom podstablu, moraju biti veće.

etzueut

```
1 def binary_search(arr, target):
2     left = 0
3     right = len(arr) - 1
4
5     while left <= right:
6         mid = (left + right) // 2
7
8         if target == arr[mid]:
9             return mid
10        elif target < arr[mid]:
11            right = mid - 1
12        else:
13            left = mid + 1
14
15    return -1
16
17 arr = [10, 20, 30, 40, 50, 60]
18 x = 50
19 print(f"Input: arr = {arr}, x = {x}")
20 print("Output:", binary_search(arr, x))
```

Input: arr = [10, 20, 30, 40, 50, 60], x = 50

Output: 4

Objašnjenje koda:

- Linija 1: Definiše se funkcija pod nazivom `binary_search` koja uzima sortiranu listu `arr` i vrednost koju treba pronaći u listi
- Linije 2 i 3: Definišu se granice opsega pretraživanja, levi indeks `left=0` (prvi element liste) i desni indeks indeks `right=len(arr)-1`, (poslednji element liste)
- Linija 5: U okviru petlje `while left<=right` proverava se da li je opseg pretraživanja [levi, desni indeks] važeći tj. da levi element liste veći od desnog

- Linija 6: Izračunava se indeks srednjeg elementa liste celobrojnim deljenjem ($//2$)
- Linija 8-13: U okviru petlje **if** poredi se prvo tražena vrednost sa vrednošću niza na srednjem indeksu **target==arr[mid]** ako su vrednosti jednake, vraća se indeks srednjeg elementa (**mid**) i izlazi se iz funkcije, ako je tražena vrednost manja od srednjeg elementa, onda traženi element mora biti u levoj polovini liste, i ažurira se desni indeks na **right=mid-1** da bi se pretraživalo u levoj polovini intervala, a ako je tražena vrednost veća od srednjeg elementa, onda je traženi broj u desnoj polovini i ažurira se levi indeks **left=mid+1** kako bi se pretraživalo u desnoj polovini intervala. Proces se nastavlja dok se ne pronađe tražena vrednost ili interval ne postane prazan (levi indeks prelazi desni indeks)
- Linija 15: Ako se tražena vrednost ne pronađe, funkcija vraća -1

Zadatak :

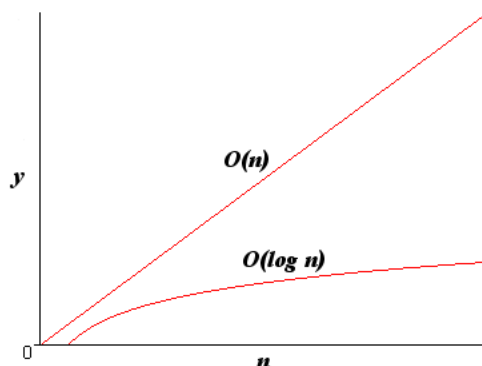
Proveriti šta je izlaz prethodnog programskog koda za sledeće slučajeve:

```
arr = [1, 2, 3, 4, 5]  
x = 3
```

```
arr = [1, 2, 3, 4, 5]  
x = 6
```

```
arr = [10, 20, 30, 40, 50, 60]  
x = 40
```

Binarni algoritam pretraživanja ima vremensku složenost $O(\log n)$, gde je n veličina ulaznog niza. U svakoj iteraciji vrši polovljenje intervala pretraživanja, što rezultira logaritamskom vremenskom složenošću. Prostorna složenost je $O(1)$ jer algoritam koristi fiksnu količinu dodatnog prostora za čuvanje levog, desnog i srednjeg indeksa, što ne zavisi od veličine ulaznog niza.



Zadatak:

Izmeniti prethodni programski kod za algoritam binarnog pretraživanja tako da se izračuna broj operacija pretraživanja tokom izvršavanja algoritma.

Izlaz da bude npr:

```
Number of operations: 8
Target 15 found at index 7
```

```
operations = 0 # Inicijalizovati brojač operacija
operations += 1 # Inkrement za svaku iteraciju u okviru petlji while i if-else
```

Primer dela koda....

```
left = 0
right = len(arr) - 1
operations = 0 # Inicijalizacija brojac operacija

while left <= right:
    operations += 1 # Increment za svaku iteraciju u petlji
    mid = (left + right) // 2

    operations += 1
    if arr[mid] == target:
        print(f"Number of operations: {operations}")
        return mid
```

Algoritam ternarnog pretraživanja

Karakteristika binarnog pretraživanja je da se pri svakom koraku deli niz koji se pretražuje na dva podniza i pomoću uslova određuje u kojem podnizu se nastavlja pretraga.

Algoritam ternarnog pretraživanja skoro potpuno se podudara s binarnim, sa tom razlikom da se umesto jedne srednje vrednosti mid biraju dve i to na sledeći način:

```
mid1 = l + (r-l)/3
mid2 = r - (r-l)/3
```

gde su *r* i *l* desna i leva granica trenutnog podniza

Zatim se upoređuje traženi broj *x* sa vrednostima `array[mid1]` i `array[mid2]`. Ako je *x* različit od obe vrednosti (nismo pronašli rešenje), postoje tri slučaja:

- 1) x je manji od $\text{array}[\text{mid1}]$: x se (ako ga ima) nalazi u intervalu $[l, \text{mid1})$
- 2) x je veći od $\text{array}[\text{mid2}]$: x se (ako ga ima) nalazi u intervalu $(\text{mid2}, r]$
- 3) inače: x se nalazi u intervalu $(\text{mid1}, \text{mid2})$

Primeri:

input_array=

8	12	25	42	57	72	85	95
0	1	2	3	4	5	6	7

traži se broj 72

1. granice opsega: left=0, right=7
2. srednje tačke:
 $\text{mid1} = \text{left} + (\text{right} - \text{left}) / 3 = 0 + (7 - 0) / 3 = 2$
 $\text{mid2} = \text{right} - (\text{right} - \text{left}) / 3 = 7 - (7 - 0) / 3 = 5$
3. poređenje key sa srednjim tačkama:
 $\text{key}(72) \neq \text{arr}[\text{mid1}](25)$
 $\text{key}(72) == \text{arr}[\text{mid2}](72)$ pronađen na indexu 5

input_array=

8	12	25	42	57	72	85	95
0	1	2	3	4	5	6	7

pronađen na indexu 5

input_array=

8	12	25	42	57	72	85	95
0	1	2	3	4	5	6	7

TRAŽI SE BROJ 8

1. granice opsega: left=0, right=7
2. srednje tačke:
 $\text{mid1} = \text{left} + (\text{right} - \text{left}) / 3 = 0 + (7 - 0) / 3 = 2$
 $\text{mid2} = \text{right} - (\text{right} - \text{left}) / 3 = 7 - (7 - 0) / 3 = 5$
3. poređenje key sa srednjim tačkama:
 $\text{key}(8) \neq \text{arr}[\text{mid1}](25)$
 $\text{key}(8) \neq \text{arr}[\text{mid2}](72)$ Key 8 nije na indeksima mid1 i mid2
4. pošto key nije na mid1 i mid2 indeksima
 proverava se da li je manji od mid1, ako jeste
 ažurira se desni indeks na mid1-1

8 je manje od 25 pa se ažurira desni indeks prvog opsega
 $\text{right} = \text{mid1} - 1$

5. izračunavaju se ponovo mid1 i mid2 za left=0 i right=1
 $\text{mid1} = 0 + (1 - 0) / 3 = 0$
 $\text{mid2} = 1 - (1 - 0) / 3 = 1$

6. proveravanje vrednosti:
 $\text{arr}[\text{mid1}] = 8 \rightarrow \checkmark$ PRONAĐEN NA INDEKSU 0

Napravi li se analogija sa binarnim pretraživanjem, problem se pri svakom koraku svodi na 1/3 prethodnog problema pa je složenost algoritma $O(\log_3(n))$

Programski kod ternarnog pretraživanja:

```
1  # Prostor pretraživanja podeljen na 3 dela:
2  #-> Prvi deo - [l,m1]
3  #-> Srednji deo - [m1,m2]
4  #-> Treći deo - [m2,r]
5
6  # l=levi index, r=desni index, m1=sredina_1 and m2=sredina_2
7
8  # Ternary Search algoritam:
9  #1. Prvo, porei se key sa elementom na mid1, ako su jednaki tada je return mid1
10 #2. Ako nisu jednaki, porei se key sa elementom na mid2, ako su jednaki tada je return mid2
11 #3. Ako nisu jednaki, proveravamo da li je key manji od elementa na mid1, ako jeste binarna pretraga prvog dela
12 #4. Ako nije, proveravamo da li je key veći od elementa na mid2, ako jeste onda binarna pretraga trećeg dela
13 #5. Ko nije, tada se binarno pretražuje srednji deo
14
15 import math as mt
16 def ternary_search(left, right, key, arr):
17     while left <= right:
18         mid1 = left + (right - left) // 3
19         mid2 = right - (right - left) // 3
20
21         # Provera da li je x=mid1
22         if (arr[mid1] == key):
23             return mid1
24
25         # Provera da li je x=mid2
26         if (arr[mid2] == key):
27             return mid2
28
29         # Key nije na mid-ovima, proverava u kojem je delu
30         if (key < arr[mid1]):
31             right = mid1 - 1
32         elif (key > arr[mid2]):
33             left = mid2 + 1
34         else:
35             left = mid1 + 1
36             right = mid2 - 1
37
38     # Key nije nadjen
39     return -1
40
41 # Driver kod
42 arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,12, 13]
43 key = 7
44 result = ternary_search(0, len(arr) - 1, key, arr)
45 if result == -1:
46     print("Element nije pronađen!")
47 else:
48     print("Element je pronađen na indeksu:", result)
49
```

Element je pronađen na indeksu: 6

Algoritam pretraživanja Jump Search

Kao i kod binarnog pretraživanja, Jump Search je algoritam pretraživanja koji se primenjuje na već uređen niz. Osnovna ideja je da se izvrši provera manjeg broja elemenata niza nego kod linearnog pretraživanja **metodom presekavanja pojedinih elemenata niza**, umesto pretraživanja svih elemenata.

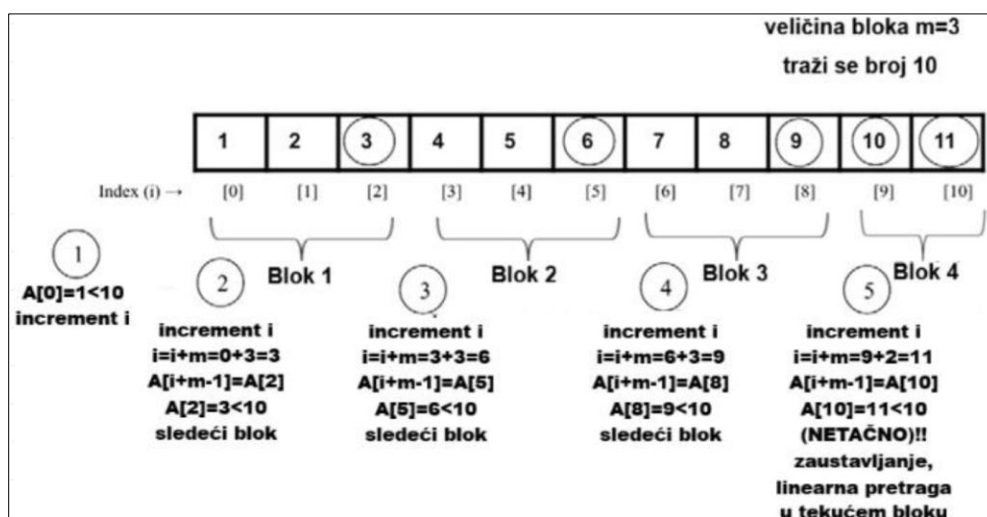
Ako pretpostavimo da imamo niz `arr[]` od `n` elemenata i jump-blok skoka veličine `m`, tada se vrši pretraživanje elemenata koji su na indeksima `arr[0]`, `arr[m]`, `arr[2m]`, ..., `arr[km]` itd. Kada se pronađe interval `arr[km] < x < arr[(k+1)m]` izvršava se operacija linearnog pretraživanja od indeksa `km` kako bi se pronašao element `x`.

Primer:

Niz [1,2,3,4,5,6,7,8,9,10]

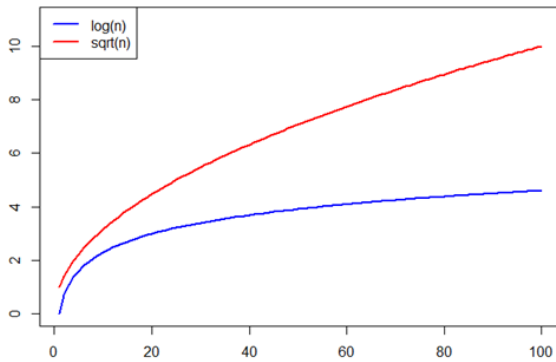
Potrebno je pronaći broj 10 u nizu.

Dužina niza je 10, veličina bloka je $m=3$ jer je $(\sqrt{10})$, postoje 4 bloka za pretraživanje



- Prvo se poredi prvi element niza sa traženim brojem 10, pošto je manji, indeks se povećava za veličinu bloka (korak 1)
- Zatim se poredi traženi broj sa poslednjim elementom svakog sledećeg bloka
- Ako je traženi broj veći, prelazi se na sledeći blok, prolazi se kroz blokove 1, 2 i 3
- U bloku 4 je traženi broj manji od poslednjeg elementa 11, zaustavlja se inkrement indeksa, vrši se linearno pretraživanje u tekućem bloku

Složenost algoritma je $O(\sqrt{n})$



Programski kod Jump Search pretraživanja (sa prikazom ukupnog broja poređenja tokom izvršavanja algoritma):

```

1  import math
2
3  def jump_search(arr, x):
4      n = len(arr)
5      step = int(math.sqrt(n))          # optimalni skok ≈ √n
6      prev = 0
7      comparisons = 0                  # brojač poređenja
8
9      # Skokovi u blokovima dok ne bude arr[min(step, n) - 1] >= x ili kraj niza
10     while prev < n and arr[min(step, n) - 1] < x:
11         comparisons += 1              # poređenje sa krajem bloka
12         prev = step
13         step += int(math.sqrt(n))
14         if prev >= n:
15             return -1, comparisons    # ako x nije pronađen
16
17     # Provera bloka u kojem bi x mogao biti
18     while prev < min(step, n):
19         comparisons += 1              # poređenja linearne pretrage
20         if arr[prev] == x:
21             return prev, comparisons
22         if arr[prev] > x:              # prekid ako se x pronađe
23             return -1, comparisons
24         prev += 1
25
26     return -1, comparisons            # nije pronađen
27
28 # Primer
29 arr = [0, 3, 5, 7, 9, 13, 17, 21, 24, 30, 35, 40, 49, 50, 77, 85, 99, 101]
30 x = 49
31 index, ops = jump_search(arr, x)
32 print("Element je na indeksu:", index)
33 print("Broj poređenja je:", ops)
34

```

Element je na indeksu: 12
Broj poređenja je: 4

Eksponencijalno pretraživanje

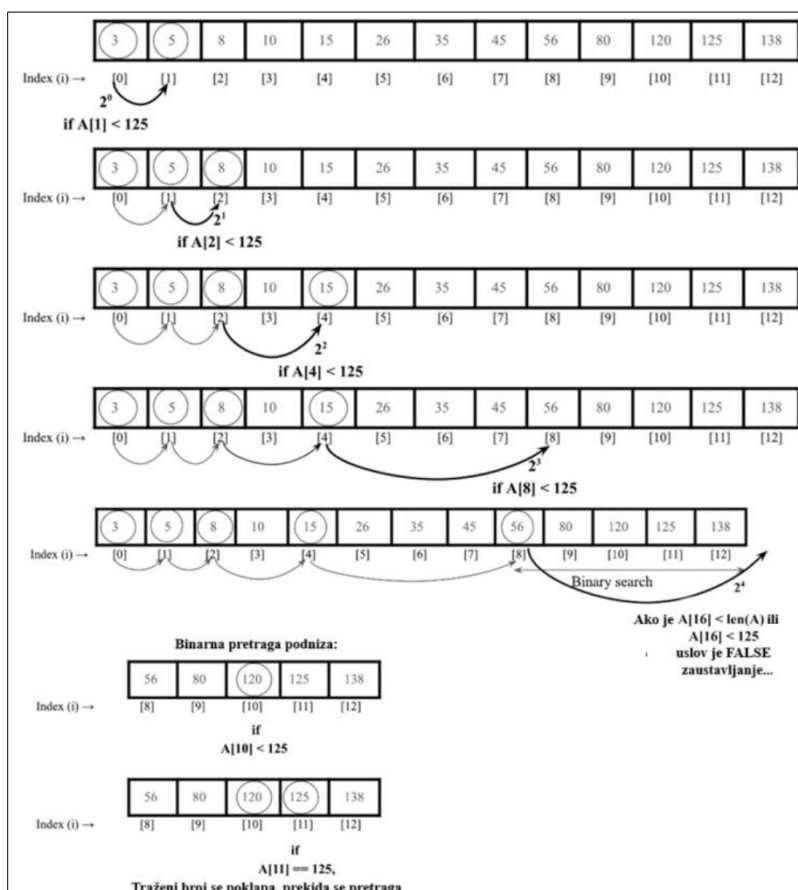
Eksponencijalno pretraživanje je još jedan algoritam pretraživanja koji se uglavnom koristi kada imamo veliki broj elemenata u nizu ili listi. Ovo pretraživanje je poznato kao „galopirajuće“ ili pretraživanje udvostručavanjem. Eksponencijalni algoritam pretraživanja se izvršava u sledeća dva koraka:

- 1) Kada je dat **sortirani niz od n elemenata**, prvo određujemo podopseg u originalnoj listi gde željeni element pretrage može biti prisutan
- 2) Zatim koristimo binarni algoritam pretrage da bismo pronašli vrednost pretrage unutar podopsega elemenata podataka identifikovanih u koraku 1

Da bismo pronašli podopseg elemenata niza, počinjemo da tražimo element u datom sortiranom nizu **preskačući 2i elemenata u svakoj iteraciji**, i je indeks elementa niza. Nakon svakog skoka, proveravamo da li je element pretrage prisutan između poslednjeg skoka i trenutnog skoka. Ako je element pretrage prisutan, onda koristimo binarni algoritam pretrage unutar ovog podniza, a ako nije prisutan, pomeramo indeks na sledeću lokaciju.

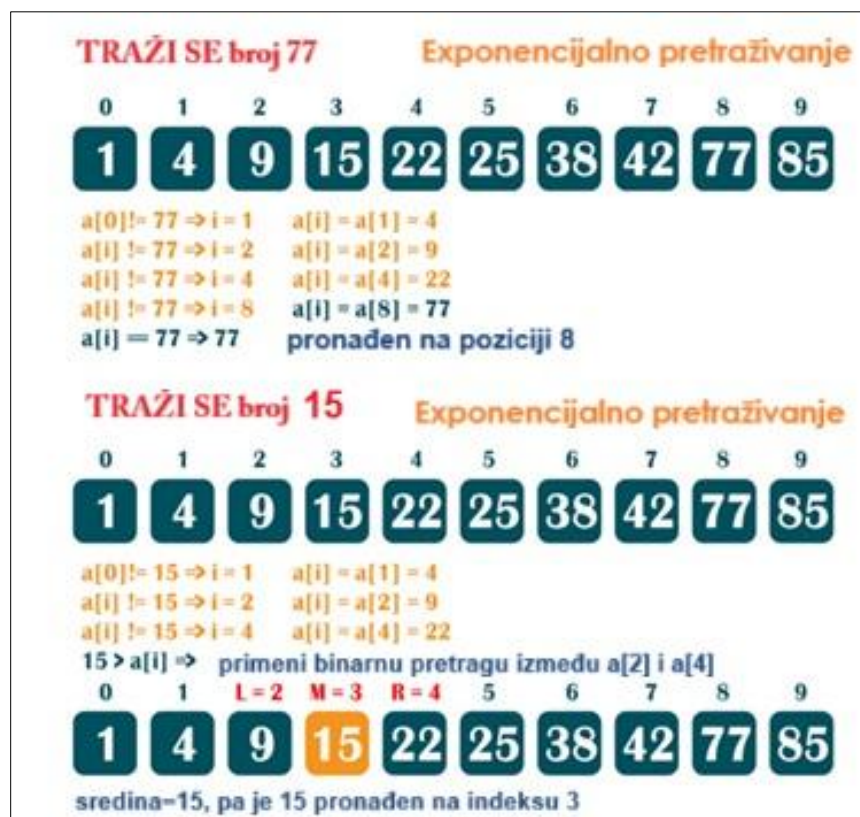
Primer:

Sortirani niz sa elementima: $A=[3, 5, 8, 10, 15, 26, 35, 45, 56, 80, 120, 125, 138]$, traži se element **125**



- Počinje se poređenjem prvog elementa niza ($i=0$, tj. $A[0]$) sa 125, pošto je $A[0] < 125$, skače se na lokaciju $2i$ sa $i=0$
- Pošto je $A[20] < 125$, uslov je **TRUE**, skače se na sledeću lokaciju sa $i=1$ tj. $A[21] < 125$
- Ponovo se skače na sledeću lokaciju $2i$ sa $i=2$, pošto je $A[22] < 125$, uslov je **TRUE**
- Iterativno skačemo na sledeću lokaciju sve dok ne završimo pretragu liste ili dok tražena vrednost ne bude veća od vrednosti na tekućoj lokaciji u listi tj. $A[2i] < \text{len}(A)$ ili $A[2i] \leq \text{tražena_vrednost}$.
- Zatim se primenjuje algoritam binarne pretrage u dobijenom podnizu

Složenost algoritma ekponencijalnog pretraživanja je $O(\log n)$



Programski kod ekponencijalnog pretraživanja:

```

1  # Python program za pronalaženje elementa x
2  # u sortiranom nizu preko Exponential Search
3  # Rekurzivna funkcija binarne pretrage vraća
4  # lokaciju x u nizu (ako postoji) a ako ne vraća -1
5
6  def binarySearch( arr, l, r, x):
7      if r >= 1:
8          mid = 1 + ( r-1 ) // 2
9
10         # Ako je x srednji element
11         if arr[mid] == x:
12             return mid
13
14         # Ako je x manji od srednjeg elementa
15         # onda je u levom podnizu
16         if arr[mid] > x:
17             return binarySearch(arr, l,
18                                 mid - 1, x)
19         # U suprotnom, x je u desnom podnizu
20         return binarySearch(arr, mid + 1, r, x)
21
22     # Ako x nije u nizu
23     return -1
24
25 # Vraća poziciju prve pojave x u nizu
26 def exponentialSearch(arr, n, x):
27     # Ako je x na prvoj lokaciji
28     if arr[0] == x:
29         return 0
30
31     # Pronađi opseg binarne pretrage udvostručavanjem indeksa 1
32     i = 1
33     while i < n and arr[i] <= x:
34         i = i * 2
35     # Poziv binarne pretrage za pronađeni opseg
36     return binarySearch( arr, i // 2,
37                           min(i, n-1), x)
38
39 # Driver kod
40 arr = [2, 3, 4, 10, 40]
41 n = len(arr)
42 x = 10
43 result = exponentialSearch(arr, n, x)
44 if result == -1:
45     print ("Element nije pronađen u nizu!")
46 else:
47     print ("Element je u nizu na indeksu: %d" %(result))
48

```

- Element je u nizu na indeksu: 3